Homework 3: Applications and Big Data

COSC 3337 — Databases — Fall 2026

Released	Oct 9, 2026
Due	Nov 1, 2026, 11:59 pm
Total	100 points, plus 5 bonus

Before you start

Databases often sit behind applications: a website or an app takes what a user clicks or types, turns it into queries, and turns the results into a page. This homework is about that layer, and about what changes when the data gets too big for one machine.

Everything uses a new database, `soundwave_app`: the Soundwave catalog you know from HW 1 (`artist`, `album`, `track`) plus the tables a streaming application needs on top of it.

Table	**Columns** (primary key underlined)
`artist`	`artist_id`, `name`, `country`, `formed_year`
`album`	`album_id`, `title`, `artist_id`, `release_year`, `genre`
`track`	`track_id`, `album_id`, `title`, `duration_sec`
`listener`	`listener_id`, `username`, `email`, `country`, `joined_date`
`playlist`	`playlist_id`, `listener_id`, `name`, `created_date`
`playlist_track`	`playlist_id`, `track_id`, `position`
`play`	`play_id`, `listener_id`, `track_id`, `played_at`

A `play` row is written every time a listener plays a track. `playlist_id` and `play_id` are `AUTO_INCREMENT`.

Part 1 is written (50 points). **Part 2** is code (50 points).

What to submit: one zip file containing exactly:

1. `hw3.pdf` — your answers to Part 1.
2. `app_queries.py` — your answers to Questions 2.1 and 2.2.
3. `mapreduce_jobs.py` — your answers to Question 2.3.

Do not rename the two `.py` files. The autograder looks for them by name.

Typesetting bonus (5 points).

- **By hand.** You may hand-write Part 1. Submit a PDF made with a scanning app, not a photograph, with every question clearly labeled.
- **With LaTeX (5 bonus points).** If you typeset Part 1 in LaTeX, you earn 5 bonus points. A template, `hw3_template.tex`, is in the `HW3` folder. Submit the compiled PDF, not the `.tex` file.

Do Part 0 first, and early, so we have time to fix any setup problems.

Part 0 — Setup *(0 points, required)*

0.1 Get the starter code

On your VM, go to your copy of the course repository, pull the new homework from `upstream` (my repository, which you set up in HW 1, Part 0.4), and load the new database:

```
cd COSC3337-FA26-homeworks
git pull upstream main
cd HW3
bash setup_db.sh
```

A plain `git pull` pulls from `origin`, your own repository, which does not have HW 3 yet. If `git pull upstream main` complains that `upstream` does not exist, run `git remote -v`: you should see both `origin` (your `cosc3337-work`) and `upstream` (mine). If `upstream` is missing, add it and pull again:

```
git remote add upstream https://github.com/isabelcachola/COSC3337-FA26-homeworks.git
git pull upstream main
```

`setup_db.sh` loads `soundwave_app`, installs the two Python packages the starter code needs (the MySQL connector and `pytest`), and checks that Python can connect. Like HW 1's script, it is safe to run again at any time and restores `soundwave_app` to a clean state, so it is your repair command. Unlike HW 1, your code *writes* to this database.

0.2 Run the tests

```
python3 -m pytest tests/ -v
```

Every test should run and *fail* with `NotImplementedError`. That means your setup works: the tests can import your files and reach the database. Any other error (cannot connect, module not found) is a setup problem. Re-run `setup_db.sh`, and if that does not fix it, email me or come to office hours.

0.3 Back up your work

Commit and push to your private `cosc3337-work` repository every time you stop working, as in HW 1. From inside `HW3`:

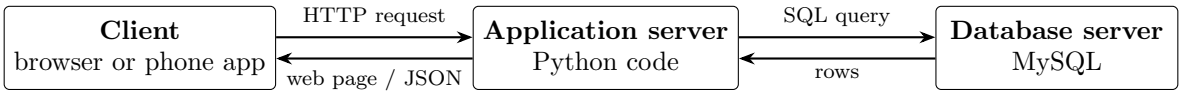
```
git add app_queries.py mapreduce_jobs.py
git commit -m <what you did this session>
git push
```

Your VM has no backup.

Part 1 — Written *(50 points)*

1.1 Application architecture *(8 points)*

Soundwave's web player is a *three-tier* application:



The functions you write in Part 2 are the application server's code.

(a) *(3 pts)* A listener taps *Play* on a track. In order, list what each of the three tiers does, from the tap to the music starting. Your list must include the SQL statement(s) the application server sends, written out in full for listener `L0042` and track `T00123`.

(b) *(3 pts)* Which tier should enforce the rule that a listener may only add tracks to *their own* playlists? Justify your answer.

(c) *(2 pts)* Give two reasons the client should *not* connect to MySQL directly.

1.2 User input and SQL *(12 points)*

Soundwave's login page runs this code. `name` is whatever the user types into the *Username* box.

```
name = input(Username: )
query = SELECT listener_id, username FROM listener WHERE username = ' + name + '
cursor.execute(query)
rows = cursor.fetchall()     # log in as the first row returned, if any
```

(a) *(3 pts)* The user types `neonfox42`. Write out the exact string that `query` holds, which is the SQL that MySQL receives.

(b) *(3 pts)* Now the user types:

```
neonfox42' OR '1'='1
```

Write out the exact SQL that MySQL receives. Which rows of `listener` does it return?

(c) *(3 pts)* Using `UNION`, write an input for the *Username* box that makes this query return *every listener's email address*. Then write out the SQL that MySQL receives.

(d) *(3 pts)* What you did in parts (b) and (c) is called **SQL injection**. The standard fix is a **parameterized query**: the SQL text contains a placeholder, `%s`, and the value is passed to `execute` separately:

```
query = SELECT listener_id, username FROM listener WHERE username = %s
cursor.execute(query, (name,))
```

What happens when your inputs from parts (b) and (c) are run through this version? Explain why it defeats both attacks.

1.3 Queries in a loop *(10 points)*

Soundwave's *My Playlists* page shows each of a listener's playlists with its number of tracks and total length. A first version of the code (`get_dashboard_slow` in `app_queries.py`) does this:

```
cur.execute(SELECT playlist_id, name FROM playlist WHERE listener_id = %s
            ORDER BY playlist_id, (listener_id,))
playlists = cur.fetchall()

for playlist_id, name in playlists:
    cur.execute(SELECT COUNT(*), SUM(t.duration_sec)
                FROM playlist_track pt JOIN track t ON pt.track_id = t.track_id
                WHERE pt.playlist_id = %s, (playlist_id,))
    ...
```

(a) *(2 pts)* How many queries does this send for a listener with N playlists?

(b) *(6 pts)* Write *one* SQL query that returns exactly what the code above returns for listener `L0042`: `(playlist_id, name, num_tracks, total_duration_sec)`, ordered by `playlist_id`.

(c) *(2 pts)* Many applications use an *ORM* (object-relational mapper), a library that lets code treat rows as ordinary objects:

```
for pl in listener.playlists:
    print(pl.name, len(pl.tracks))
```

How many queries might this code send for a listener with N playlists? Compare it with `get_dashboard_slow`: which version makes the cost easier to see, and why?

1.4 Scaling out: sharding *(10 points)*

Soundwave's `play` table has grown to billions of rows, too many for one server, so it is *sharded*: split across 4 servers ("nodes"), each holding some of the rows. Two schemes are proposed:

- **Scheme H (hash):** a row goes to node *hash*(`listener_id`) mod 4.
- **Scheme R (range):** a row goes to a node by `played_at`: node 1 holds January–March 2026, node 2 April–June, node 3 July–September, node 4 October–December.

(a) *(6 pts)* For each query below and each scheme, how many nodes must be contacted? Justify each of the six answers.

1. Listener `L0042`'s full play history (*Recently played*).
2. The number of plays of each track during March 2026.
3. Inserting the row for a play that is happening right now.

(b) *(4 pts)* A *hot spot* is a node that receives far more work than the others. Does either scheme create hot spots? Explain.

1.5 MapReduce by hand *(10 points)*

MapReduce computes over data spread across many machines in three steps:

1. **Map:** a function you write is called on each input record, on whichever machine holds it, and emits zero or more *(key, value)* pairs.
2. **Shuffle:** the framework collects all pairs with the same key onto the same machine, producing *key* → *[values]*.
3. **Reduce:** a second function you write is called once per key, on that key's list of values, and emits the result for that key.

Here are ten rows of the play log (with each track's genre already attached):

play_id	listener	track	genre	play_id	listener	track	genre
1	L01	T05	Folk	6	L01	T11	Rock
2	L02	T11	Rock	7	L03	T09	Indie
3	L01	T07	Folk	8	L02	T02	Indie
4	L03	T02	Indie	9	L01	T05	Folk
5	L02	T05	Folk	10	L03	T13	Rock

The goal is the number of plays per genre.

(a) *(2 pts)* Write the *(key, value)* pair the map function emits for each of the ten rows.

(b) *(2 pts)* Write the output of the shuffle.

(c) *(2 pts)* Write the output of each reduce call.

(d) *(2 pts)* Write the SQL query that computes the same thing from the `soundwave_app` tables. Which part of your query corresponds to the shuffle?

(e) *(2 pts)* Suppose the play log is split across 100 machines. Describe how the work of each step is divided among them and what, if anything, has to move between machines. Why does this make MapReduce suitable for data too big for one database server?

Part 2 — Code *(50 points)*

The files

File	What it is	You...
`app_queries.py`	2.1 and 2.2	**edit and submit**
`mapreduce_jobs.py`	2.3	**edit and submit**
`db.py`	`get_connection()` and `QueryCounter`	use, don't edit
`mapreduce_driver.py`	`run_mapreduce()` and `read_plays()`	use, don't edit
`data/plays.csv`	the `play` table exported with `artist_id` and `genre`	read
`tests/`	the public tests	run

Every function you write has a docstring that is its full specification: what it receives, what it returns, in what order, and with examples. Read it before you start the function. **Do not rename functions or change their parameters**; the autograder calls them exactly as written.

How grading works

The autograder runs the public tests in `tests/` *plus* hidden tests for edge cases, on a **different copy of the data** (different listeners, playlists and plays). Passing every public test means your functions have the right shape. It does not guarantee full marks.

Python and MySQL

```
from db import get_connection
conn = get_connection()                          # autocommit is OFF
cur = conn.cursor()
cur.execute(SELECT name, formed_year FROM artist
            WHERE country = %s AND formed_year > %s, (Spain, 2010))
for name, year in cur.fetchall():                # a list of tuples
    print(name, year)
cur.close()
```

- Every value from outside the program goes in through a `%s` placeholder and the second argument of `execute`, which is always a tuple. Never put quotes around `%s`: the connector handles quoting. To match part of a string, put the wildcards in the value: `cur.execute(... LIKE %s, (% + text + %,))`.
- `fetchall()` returns a list of tuples; `fetchone()` returns one tuple, or `None`.
- After an `INSERT` into a table with an `AUTO_INCREMENT` key, `cur.lastrowid` holds the key MySQL just assigned.

Commit and rollback

With autocommit off, every change you make is *provisional*. Your own connection can see it, but no one else can, and it is not saved.

- `conn.commit()` makes every provisional change since the last commit permanent and visible to everyone.

- `conn.rollback()` throws every provisional change since the last commit away, as if it never happened.

This is how you make several statements *all or nothing*: do them all, then commit; if any one fails, roll back. When a statement violates a primary key or foreign key, the connector raises `mysql.connector.IntegrityError`, which you can catch:

```
import mysql.connector
try:
    ...
except mysql.connector.IntegrityError:
    ...
```

2.1 Database-backed functions *(20 points)*

In `app_queries.py`, write these three functions. Use parameterized queries for every one.

- `get_top_songs(conn, artist_id, k)` — an artist's `k` most-played tracks as `(track_id, title, play_count)`. Tracks that were never played still count, with 0.
- `create_playlist(conn, listener_id, name, track_ids)` — create a playlist and its tracks and return the new `playlist_id`. On any bad input, raise `ValueError` and leave no rows behind.
- `search_songs(conn, text)` — every track whose title contains `text`, as `(track_id, title, artist_name)`. `text` is untrusted user input; implement your function with protections against SQL injection (see Question 1.2).

What is tested	Points
`get_top_songs`: correct rows and order for several artists and values of `k`	4
`get_top_songs`: ties, `k` larger than the catalog, never-played and unknown artists	2
`create_playlist`: valid input creates the rows, commits, returns the new id	4
`create_playlist`: bad input raises `ValueError` and *no* rows remain	4
`search_songs`: correct matches, case-insensitive, including titles with apostrophes	3
`search_songs`: inputs like those in Question 1.2 return no extra rows and do not crash	3

2.2 Speed up the dashboard *(10 points)*

`get_dashboard_slow` (Question 1.3) is provided and correct. Write `get_dashboard(conn, listener_id)` so that it returns *exactly* the same thing for every listener, using *at most 2 queries*, however many playlists the listener has.

`db.QueryCounter` wraps a connection and counts the queries sent through it. The autograder uses it to check you, so you can check yourself first:

```
from db import get_connection, QueryCounter
qc = QueryCounter(get_connection())
get_dashboard(qc, L0001)
print(qc.count, qc.queries)
```

What is tested	Points
Output identical to `get_dashboard_slow`, including listeners with no playlists and empty playlists	5
At most 2 queries (*only awarded if the output is also correct*)	5

2.3 MapReduce in Python *(20 points)*

`mapreduce_driver.py` contains a tiny single-machine MapReduce, `run_mapreduce(records, map_fn, reduce_fn)`. It calls `map_fn` on every record, does the shuffle, and calls `reduce_fn` once per key, exactly the three steps from Question 1.5. You write only the map and reduce functions, in `mapreduce_jobs.py`.

- **Job 1, plays per genre.** `genre_map` and `genre_reduce`. This is Question 1.5 in code.
- **Job 2, each listener's most-played artist.** This needs *two* MapReduce stages, because counting plays needs one grouping (by listener and artist) and picking the maximum needs another (by listener). Stage 1 is `count_map`/`count_reduce`; Stage 2 is `top_map`/`top_reduce`, and its input records are Stage 1's output pairs. `top_artist_job` chains them. Ties go to the smaller `artist_id`.

The public tests check both jobs against the same question asked in SQL, so when they pass, MapReduce and SQL agree.

Rules for Question 2.3 (checked by the autograder)

- `mapreduce_jobs.py` may not import `pandas`, `numpy`, `sqlite3`, `mysql`, `pymysql` or `collections`.
- A mapper sees one record; a reducer sees one key and its values. Do not group values yourself; that is the shuffle's job.

What is tested	Points
`genre_map` emits the right pair	2
`genre_reduce` returns the right pair	2
Job 1 on the full play log equals the SQL `GROUP BY` answer	4
Stage 1 of Job 2 (plays per listener and artist) is correct	3
`top_map` and `top_reduce` are correct, including ties	3
`top_artist_job` on the full play log equals the SQL answer	4
No forbidden imports (*only awarded if Job 1 is implemented*)	2

Before you submit

1. `python3 -m pytest tests/ -v` passes.
2. You tested edge cases the public tests do not cover.
3. Your zip contains `hw3.pdf`, `app_queries.py` and `mapreduce_jobs.py`, with those exact names.